Programming Language Implementation
– practical assignment –

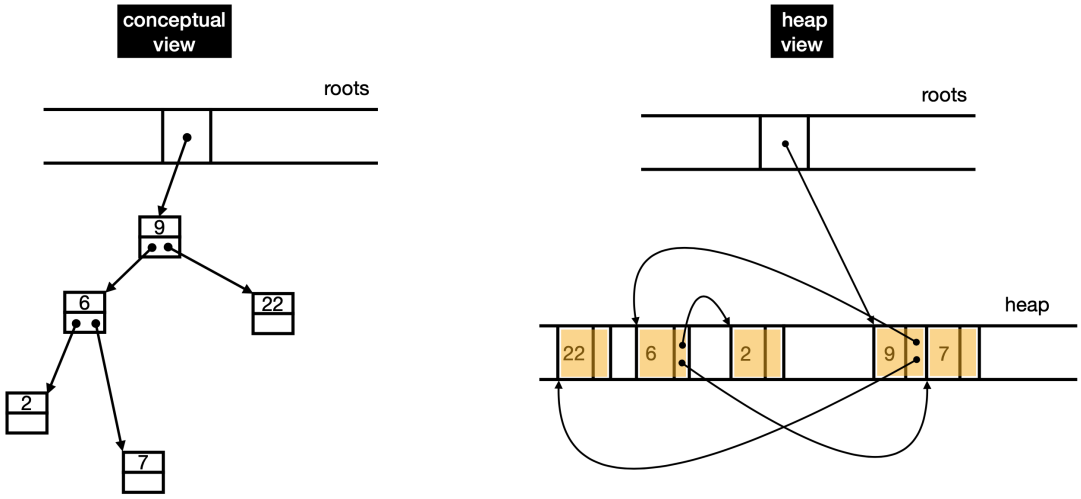
---

1. Consider the file `gc-code.zip` available on the course's Moodle page. The code provided implements a simple application that can serve as a basis for implementing *garbage collection* algorithms. Download and extract the file, inspect the contents of the resulting directory, and follow the instructions in the `readme` file:

```
$ unzip gc-code.zip
$ cd gc-code
$ ls
bistree.c
bistree.h
bool.h
collector.c
collector.h
globals.h
heap.c
heap.h
list.c
list.h
makefile
mutator.c
readme
```

The file `mutator.c` contains the `main()` function and runs a loop in which a set of predefined binary trees is modified. At each iteration, the code randomly decides whether to add/delete a random integer to/from an existing tree. The decision is based on a parameter `threshold`, a number between 0 and 100, passed to the `main()` function.

The roots of the trees are stored in an array `roots`. The binary search tree is defined and implemented in `bistree.h` and `bistree.c`. There is also an implementation for lists that will be useful, in `list.h` and `list.c`.
In this implementation, memory for tree nodes is allocated using the function `my_malloc()`, which is called in `bistree.c`. This function allocates space in the heap implemented in `heap.h` and `heap.c`.



Note that each block of bytes allocated in the heap is preceded by a *header* that stores the block size and whether it is in use, i.e., whether it is reachable from one of the roots in `roots`. This information, invisible to the programmer, is crucial for the collector.

```
typedef struct {
   unsigned int marked;
   unsigned int size;
} _block_header;
```

Associated with the heap are functions that implement collectors, which must be implemented in `collector.c` and whose types are defined in `collector.h`. Note that during heap initialization, one of these functions is passed as an argument.

Also note the `makefile` included with the code. Compile and test the program:

```
$ make -f makefile all
...
/* add 50%, delete 50%, 100 roots, 1000000 iterations */
$ ./mutator 0.5 100 1000000
/* add 90%, delete 10%, 100 roots, 1000000 iterations */
$ ./mutator 0.1 100 1000000
/* add 10%, delete 90%, 100 roots, 1000000 iterations*/
$ ./mutator 0.9 100 1000000
```

2. Based on the previous code and the description of algorithms found in the recommended book (Jones, Hosking, and Moss), write code for the following collectors: Mark&Sweep, Mark&Compact, and Copy-Collection. Empty prototypes are provided in `collector.c`:

```
void mark_sweep_gc(List* roots) { ... }
void mark_compact_gc(List* roots) { ... }
void copy_collection_gc(List* roots) { ... }
```

If possible, keep the provided API unchanged. Observe the behavior of the algorithms during execution of the `mutator` program for different values of `threshold`. In particular, note the fraction of the execution time spent on GC.

3. Now consider, instead of the program `mutator.c`, a specific language to execute operations on trees. The following example, in a *toy language*, performs 100 random add/delete operations on a predefined set of trees:

```
   llp  100
__loop:
   rnd  20
   sel
   rnd  100
   blt  __add
   del
   j    __end
__add:
   add
__end:
   jlp  __loop
   quit
```

where:

instruction	description	op
`llp n`	loads loop-counter with `n`	0x1
`jlp L`	decrements `loop-counter`, jumps to `L` if not 0	0x2
`j L`	jumps to `L`	0x3
`blt L`	jumps to `L` if `Stack[top]` $<$ `threshold`	0x4
`rnd n`	generates integer up to `n`, stores it in `Stack[top]`	0x5
`sel`	generates integer up to `roots-size`, stores it in `Stack[top]`	0x6
`add`	adds integer `Stack[top-1]` to tree `Roots[Stack[top-2]]`	0x7
`del`	deletes integer `Stack[top-1]` from tree `Roots[Stack[top-2]]`	0x8
`quit`	exits the program	0x9

with `loop-counter`, `roots-size` and `threshold` being parameters. The above program can be written in bytecode (2 bytes per instruction):

```
#define PAD 0x00

byte VM_program[] = {
   0x01, 0xc4,
   0x05, 0x14,
   0x06, PAD,
   0x05, 0xc4,
   0x04, 0x0e,
   0x08, PAD,
   0x03, 0x10,
   0x07, PAD,
   0x01, 0x02,
   0x02, 0x02,
   0x09, 0x00
}
```

Programs in this toy language can be executed by a Toy Virtual Machine (ToyVM) composed of:

a) `int VM_stack[]` – stack for integer values;

b) `byte VM_heap[]` – heap for storing trees;

c) `byte VM_program[]` – program to execute;

d) `tree* VM_roots[]` – roots of trees stored in the heap;

e) an emulator loop.

The following code (call it `toyvm.c`) can be used as a skeleton for the emulator loop:

```
int main(int argc, char* argv[] ) {

int    VM_threshold;
int    VM_loop_counter;
int    VM_roots_size;
int    VM_stack_size;
int    VM_stack_top;
int    VM_stack[];
byte   VM_heap[];
byte   VM_program[];
tree   VM_roots[];
```

```
/* initialize threshold */
VM_threshold = atoi(argv[1]);

/* initialize roots */
VM_roots_size = atoi(argv[2]);
VM_roots      = (tree*)malloc(VM_roots_size * sizeof(tree));
for(int i = 0; i < roots_size; i++)
  VM_roots[i] = new_empty_tree();

/* initialize stack */
VM_stack_size = atoi(argv[3]);
VM_stack      = (int*)malloc(VM_stack_size * sizeof(int));
VM_stack_top  = 0;

/* initialize heap */
VM_heap = ...;

/* initialize program */
#define PAD  0x00

byte VM_program[] = {
   0x01, 0xc4,
   0x05, 0x14,
   0x06, PAD,
   0x05, 0xc4,
   0x04, 0x0e,  /* __add = 14 = 0x0e */
   0x08, PAD,
   0x03, 0x10,  /* __end = 16 = 0x10 */
   0x07, PAD,
   0x01, PAD,
   0x02, 0x02,  /* __loop = 2 = 0x02 */
   0x09, 0x00
}

/* run program */
byte* pc = VM_program;
for( ;  ; ) {
   byte  opcode = pc[0];
   switch (opcode) {
   case 0x01:
      /* llp */
      VM_loop_counter = pc[1];
```

```
   pc = pc + 2;
   break;
case 0x02:
   /* jlp */
   VM_loop_counter--;
   if ( VM_loop_counter != 0 )
      pc = &VM_program[pc[1]];
   else
      pc = pc + 2;
   break;
case 0x03:
   /* j */
   pc = &VM_program[pc[1]];
   break;
case 0x04:
   /* blt */
   if ( VM_stack[--VM_stack_top] < VM_threshold )
      pc = &VM_program[pc[1]];
   else
      pc = pc + 2;
   break;
case 0x05:
   /* rnd */
   VM_stack[VM_stack_top++] = rand() % pc[1];
   pc = pc + 2;
   break;
case 0x06:
   /* sel */
   value_stack[VM_stack_top++] = rand() % roots_size;
   pc = pc + 2;
   break;
case 0x07:
   /* add */
   int  number  = VM_stack[VM_stack_top - 1];
   tree* root   = &VM_roots[VM_stack[VM_stack_top - 2]];
   bstree_insert(number, root);
   VM_stack_top = VM_stack_top - 2;
   pc = pc + 2;
   break;
case 0x08:
   /* del */
   int  number  = VM_stack[VM_stack_top - 1];
   tree* root   = &VM_roots[VM_stack[VM_stack_top - 2]];
```

```
      bstree_delete(number, root);
      VM_stack_top = VM_stack_top - 2;
      pc = pc + 2;
      break;
   case 0x09:
      /* quit */
      exit(0);
   default:
      /* error, exit */
      printf("%x: unkown opcode\n");
      exit(1);
   }
  }
}
```

Correct and extend this code so that it works with your heap and garbage-collection implementation, forming a complete implementation of ToyVM.

4. The code from parts 2 and 3 must be submitted to `lmlopes@fc.up.pt` as part of a `.zip` file containing the original code. You must also include a `readme` file describing how to test the algorithms. The deadline is 23:59 UTC+1, May 29th. The assignment is individual and must be presented by the student to receive a grade according to:

- 1 mandatory in-class progress report (30%) until May 15th — students unable to attend may schedule another time or use Zoom;
- final code and presentation (70%).

The progress report is informal: the student explains the current status of the work, the difficulties he encountered, and explains the available code. Presentations will take place during the week of June 1–5, at a scheduled time.